Scoped Enumeration Stringification and Annotation-Driven Defaultable Formatting with C++26 Static Reflection

Extensions for `std::to_string` and `std::formatter`

Document #:	D0000R0 [Latest] [Status]
Date:	2026-11-08
Project:	Programming Language C++
Audience:	Library Evolution Working Group (LEWG)

Reply-to:	Jeremy Murphy <jeremy.r.murphy at protonmail dot com>

Contents

1 Abstract **1**

2 Revision History **1**
2.1 R0 1

3 Motivation and Scope **1**
3.1 Before / After Comparison 1

4 Design Decisions **2**
4.1 Alternatives Considered 2

5 Proposed Wording **2**

6 Implementation Experience **2**

7 References **2**

1 Abstract

A short 1–2 paragraph overview explaining the proposal, the problem it solves, and the basic approach being proposed for C++.

2 Revision History

2.1 R0

— Initial revision.

3 Motivation and Scope

Explain why this proposal exists, what real-world use cases it addresses, and the limitations of current C++.

3.1 Before / After Comparison

Before (Current C++)	After (With Proposal)

`// Motivating example showing current verbose`
`// error-prone, or inefficient code`
`#include <iostream>`
 	
`void legacy_way() {`
`  // Current approach`
`}`
	`// Clean code using your proposed feature`
	`void proposed_way() {`
	`  // Proposed syntax or library feature`
	`}`

`void legacy_way() {`
`  // Current approach`
`}`
	`void proposed_way() {`
	`  // Proposed syntax or library feature`
	`}`

4 Design Decisions

Explain the technical design choices, trade-offs, ABI implications, and feature interactions.

4.1 Alternatives Considered

Option A	Option B
`//chose this one`	⃠ **Not proposed** `//didn't do this`

5 Proposed Wording

Draft edits relative to the C++ Working Draft. Use markup tags like `[added text]{.add}`, `[removed text]{.rm}`, and `[removed text](added text){.sub}`.

Modify Section [general.description] as follows:

The behavior of the function shall be modified to support the new functionality ~~throw an exception~~.

6 Implementation Experience

Note whether a prototype exists, compiler branches where it has been implemented, or user experience gained from library implementations.

7 References